matrix multiplication explanation

13 May 2026 01:50

HPC Assignment 7 — Parallel Matrix Multiplication using OpenMP

This assignment performs:

Parallel Matrix Multiplication

using:

- OpenMP
- nested loops
- data parallelism

1. What We Have To Do?

We have to:

1. input two matrices
2. check multiplication condition
3. multiply matrices
4. perform multiplication in parallel using OpenMP
5. display resultant matrix

Main Goal

To demonstrate:

parallel computation using matrix multiplication

2. What is Matrix Multiplication?

Matrix multiplication combines:

rows of first matrix

with:

columns of second matrix

Multiplication Condition

MOST IMPORTANT.
Matrix multiplication possible only if:
Columns\ of\ A = Rows\ of\ B

Example

Matrix A:
2 × 3
Matrix B:
3 × 2
Multiplication:

possible

Result matrix:
2 × 2

3. Matrix Multiplication Formula

Each element:

$$C[i][j]=\sum_{k=0}^{n-1}A[i][k]\times B[k][j]$$

MOST IMPORTANT FORMULA.

Meaning

To compute:

C[i][j]

Take:

- row of A
- column of B

Multiply corresponding elements and add them.

Example

A:

1 2

3 4

B:

5 6

7 8

Calculate First Element

C[0][0]

=

$$(1\times5)+(2\times7)=19$$

Calculate Second Element

C[0][1]

=

$$(1\times6)+(2\times8)=22$$

Final Result

19 22

43 50

4. Overall Workflow

Input Matrix A

↓

Input Matrix B

↓

Check Compatibility

↓

Parallel Matrix Multiplication

↓

Store Result in Matrix C

↓

Display Result

5. Header Files

```
#include <iostream>
#include <omp.h>
```

Purpose

Header	Purpose
iostream	input/output
omp.h	OpenMP support

6. Input Matrix Dimensions

int rowsA, colsA, rowsB, colsB;
Stores dimensions of matrices.

User Input

cin >> rowsA >> colsA;
cin >> rowsB >> colsB;

Example

If user enters:
2 2
2 2
Then:

- A is 2×2
- B is 2×2

7. Compatibility Check

MOST IMPORTANT.
if (colsA != rowsB)

Why?

Because matrix multiplication rule requires:
colsA=rowsB

If Condition Fails

return 0;
Program stops.

8. Dynamic Memory Allocation

```
int *A = new int[rowsA * colsA];
int *B = new int[rowsB * colsB];
int *C = new int[rowsA * colsB];
```

Purpose

Creates:

- matrix A
- matrix B
- result matrix C

Why 1D Array Used?

Instead of:
2D arrays
program uses:

flattened 1D representation

for easier dynamic allocation.

Memory Layout Example

Matrix:
1 2
3 4
Stored as:
[1 2 3 4]

Access Formula

Element:
A[i][j]=A[i\times cols+j]
MOST IMPORTANT INDEXING CONCEPT.

Example

For:
A[1][0]
Formula:
1\times2+0=2
So:
A[2]
correctly accesses:
3

9. Input Matrix Elements

```
for (int i = 0; i < rowsA * colsA; i++)
   cin >> A[i];
```

Reads matrix A.
Similar loop for matrix B.

10. MOST IMPORTANT PART — Parallel Matrix Multiplication

Code:
#pragma omp parallel for
MOST IMPORTANT LINE.

What Does It Do?

Parallelizes:

outer row loop

Different threads process different rows simultaneously.

Outer Loop

```
for (int i = 0; i < rowsA; i++)
```

Each thread handles:

one row of result matrix

Middle Loop

for (int j = 0; j < colsB; j++)
Processes columns.

Inner Loop

for (int k = 0; k < colsA; k++)
Performs:

dot product calculation

Sum Variable

int sum = 0;
Stores:

one matrix cell result

Core Multiplication

sum += A[i * colsA + k] * B[k * colsB + j];
MOST IMPORTANT FORMULA.
Implements:
C[i][j]+=A[i][k]\times B[k][j]

Store Result

C[i * colsB + j] = sum;
Stores computed element.

11. Why Parallelism Works Well Here?

Because:

different rows are independent

Example:
Thread 1 computes:
Row 0
Thread 2 computes:
Row 1
No conflicts occur.

12. Output Matrix

cout << C[i * colsB + j];
Displays resultant matrix.

Example Output

19 22
43 50

13. Memory Deallocation

```
delete[] A;
delete[] B;
delete[] C;
```

Why Needed?

Prevents:

memory leaks

14. Example Execution

Input:
Matrix A:
1 2
3 4
Matrix B:
5 6
7 8

Parallel Work Distribution

Thread 1:
computes row 0
Thread 2:
computes row 1

Result

19 22
43 50

15. Time Complexity

Sequential Matrix Multiplication:
$O(n^3)$

Parallel Matrix Multiplication

Approximate:
$O\left(\frac{n^3}{P}\right)$
Where:

- (P) = processors/threads

Why Faster?

Because:

- rows processed simultaneously

16. Advantages of Parallel Matrix Multiplication

Advantage	**Explanation**
Faster computation	multicore utilization
Good parallelism	row independence
Widely used	AI, graphics, HPC

17. Important Viva Questions

Q1. What is matrix multiplication?

Combining rows of first matrix with columns of second matrix.

Q2. What is condition for matrix multiplication?

colsA=rowsB

Q3. What does #pragma omp parallel for do?

Parallelizes loop iterations across threads.

Q4. Why matrix multiplication suitable for parallelism?

Because different rows/elements can be computed independently.

Q5. Why 1D arrays used for matrices?

For dynamic memory allocation and simpler memory handling.

Q6. What is dot product?

Multiplication and summation of row and column elements.

Q7. What is time complexity of matrix multiplication?

Sequential:
O(n^3)
Parallel:
O\left(\frac{n^3}{P}\right)

18. Short Practical Explanation

> This program performs parallel matrix multiplication using OpenMP. Matrix multiplication is implemented using nested loops, where different rows of the result matrix are computed simultaneously using #pragma omp parallel for. Matrices are stored using dynamically allocated 1D arrays, and the result matrix is generated using dot product calculations.

From <https://chatgpt.com/c/6a030630-1bdc-83ec-a58f-b5d0dcf67e97>